

연구 개요

과제 선정 배경

- 생성형 AI 확산으로 코드 생성량과 개발 속도는 증가하지만, 보안 검증 부담도 함께 증가
- AI가 생성한 코드에서도 다양한 보안 취약점이 발생하여 개발 단계의 자동 보안 검증 필요성 증가
- 소스코드의 복잡한 호출 관계와 데이터 흐름까지 분석할 수 있는 구조적 보안 분석 기술 필요

기존 접근의 한계

- 기존 SAST의 고정 규칙만으로는 메서드 간 데이터 흐름과 미등록 외부 API 탐지에 한계
- LLM 단독 분석은 코드의 의미를 이해할 수 있지만 정확한 취약 경로 제시와 결과 재현이 어려움
- 정적 분석은 의미 이해가 부족하고, LLM은 구조적 근거가 부족하여 각각의 장단점이 존재

과제 목표

- Joern CPG를 활용해 Java 코드의 구조와 source-to-sink 데이터 흐름을 분석
- 고정 규칙에서 놓치는 외부 API는 Qwen3.8-Max의 의미 분석으로 source/sink 역할을 보완
- CPG의 구조적 근거와 LLM의 의미 판단을 결합한 Java 취약점 탐지 엔진 구축
- 대규모 벤치마크 평가와 실제 서비스 연동을 통해 탐지 성능과 실제 적용 가능성을 검증

평가 설계

평가 데이터

534쌍

취약 코드 ↔ 안전 코드

- NIST Juliet Java 1.3기반
- CWE의 취약/안전 50:50
- 중복 제거 후 534쌍으로 구성

비교 대상

4개

평가 모델·규칙

- CPG + 사전 정의 규칙
- CPG + 사전 정의 규칙 + LLM 생성 규칙
- Qwen 단독 소스코드 분석
- CodeQL (SAST)

취약점 범위

38종

CWE 유형

7종

- 규칙 기반 탐지
CPG 규칙으로 찾는 취약점

14종

- 데이터 흐름 추적
입력값이 위험한 지점까지 전달되는지 추적이 필요한 취약점

17종

- 탐지 범위 외
CPG 고정 탐지 규칙이 미구현된 취약점

평가 지표

5가지

Precision(정밀도)

탐지한 것 중 실제 취약점 비율

Recall(재현율)

실제 취약점 중 찾아낸 비율

F1(종합 성능)

정밀도와 재현율의 균형

FPR(오탐률)

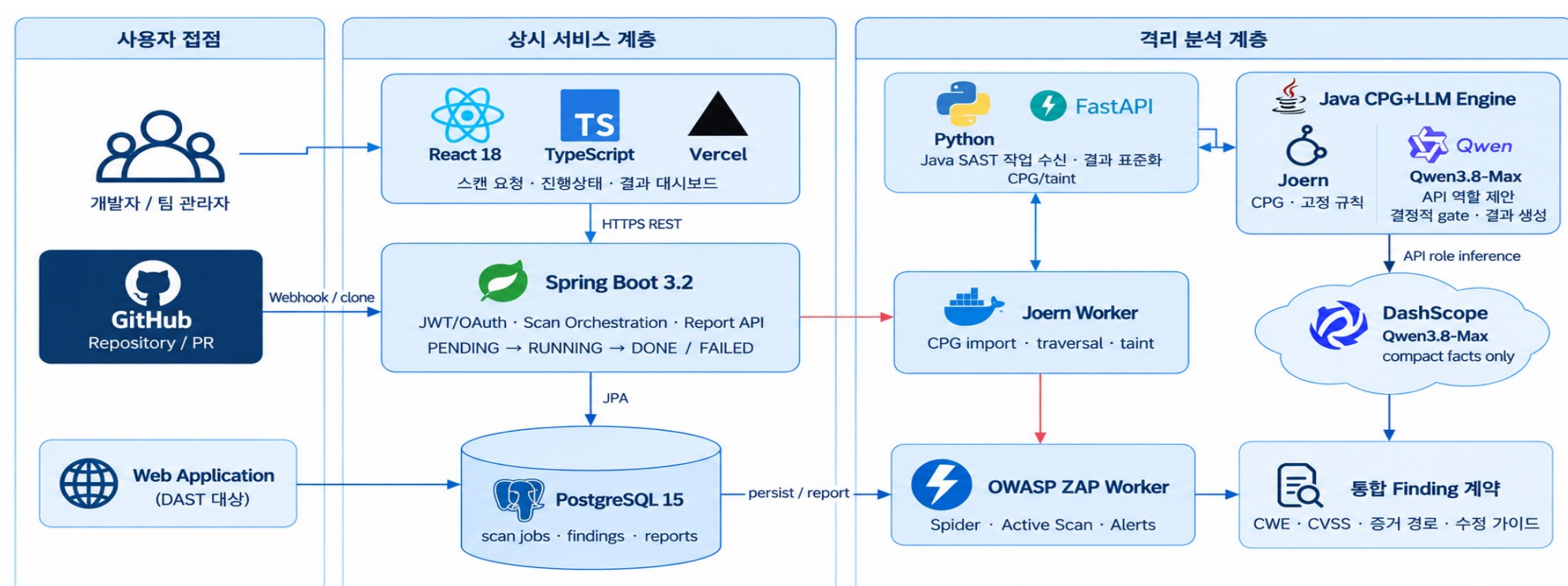
정상 코드를 취약하다고 판단한 비율

MCC(상관계수)

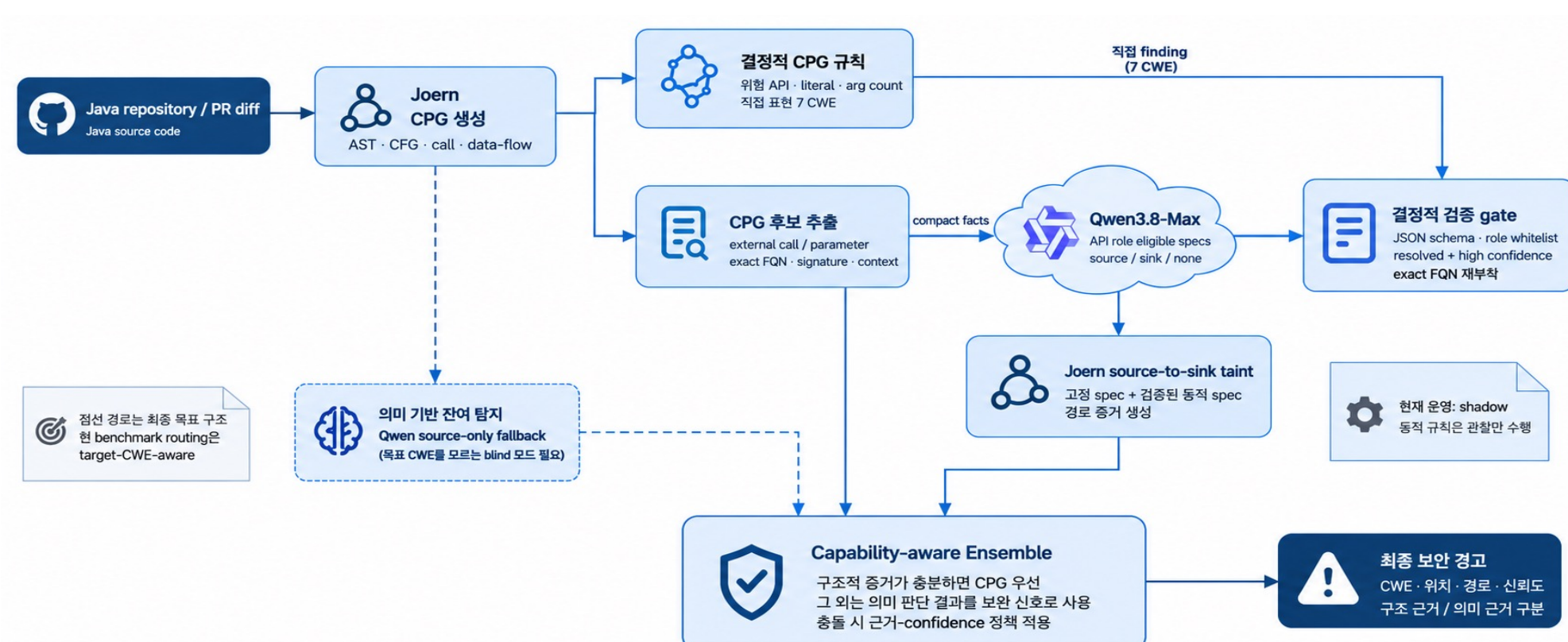
전체 판정의 신뢰도

핵심 기술 및 구현

전체 서비스 아키텍처



Java CPG+LLM 탐지 모델



데이터 준비부터 탐지 결과 확인까지

01 데이터셋 준비

평가용 코드 수집·전처리

NIST Juliet Java 1.3
중복 제거 · 답 표지 중립화
개발 190쌍 / 평가 534쌍
평가 대상: 취약점 38종

같은 유형의 취약·안전 코드를
짝지어 동일 조건으로 평가

02 CPG · Joern

코드 구조와 API 후보 추출

```
"id": 17  
"bare_name": "execute"  
"resolved": true  
"external": true
```

함수 호출·주변 코드를 추출하고
입력에서 위험 동작까지 추적

03 Qwen3.8-Max

API 역할 제언과 의미 검토

```
"id": 17  
"role": "sink"  
"cwe": "CWE-89"  
"endpoint": "arg:1"
```

평가: 입력·위험 함수 규칙 제안
서비스: 소스코드 의미 검토

04 결과 통합

검증된 근거로 결과 정리

평가: 규칙 검증·추가 후
CPG 경로 탐지
서비스: CPG + 의미 탐지
CWE·행 번호로 중복 통합

서비스는 신뢰도 높은 의미 탐지와
CPG 경로를 함께 보존

05 서비스 결과 화면



실험 결과 및 결론

전체 및 취약점 유형별 탐지 성능

평가 대상	적용한 탐지 방식	정밀도	탐지율	F1	오탐률	MCC
전체 취약점 38종	기존 CPG 규칙	77.4%	27.0%	0.400	7.9%	0.252
	기존 CPG 규칙 + LLM 생성 규칙	72.6%	34.3%	0.466	12.9%	0.251
	LLM 단독(대상 CWE 사전 제공)	82.6%	95.1%	0.884	20.0%	0.760
	CodeQL	74.2%	21.5%	0.334	7.5%	0.199
CPG 규칙으로 찾는 취약점 7종	기존 CPG 규칙	100.0%	100.0%	1.000	0.0%	1.000
	기존 CPG 규칙 + LLM 생성 규칙	100.0%	100.0%	1.000	0.0%	1.000
데이터 흐름 추적이 필요한 취약점 14종	기존 CPG 규칙	50.0%	20.0%	0.286	20.0%	0.000
	기존 CPG 규칙 + LLM 생성 규칙	57.9%	34.8%	0.435	25.2%	0.104
CPG 고정 규칙 미구현 취약점 17종	기존 CPG 규칙	탐지 없음	0.0%	0.000	0.0%	0.000
	기존 CPG 규칙 + LLM 생성 규칙	33.3%	3.6%	0.065	7.2%	-0.080

동일한 평가 대상 안에서 두 방식의 점수를 비교. 전체 38종은 위 네 방식으로 비교.
17종의 기존 CPG는 탐지 결과 없음. MCC는 분모가 0이면 평가 코드 기준 0으로 처리.

결론과 향후 계획

- CPG의 규칙 탐지와 LLM 보강의 탐지율 향상
- 미탐 원인을 규칙 누락·흐름 추적 실패로 분류, LLM으로 외부 API 규칙 후보 생성, CPG로 취약 경로 검증
- Java 외 언어로 지원 확대, 실제 취약점 사례로 검증 예정